Forge IDE UI Architecture Report

Recommendation: add a Rust owned browser UI to the existing read only agent. Use a stable generated shell, a small browser adapter and a same origin native host; do not replace the architecture with React.

Implemented foundation

forge-agent selects named nodes and permitted project aliases. Its mTLS gateway invokes the shared bridge and returns a typed structural **ProjectReport**.

Localhost two-process evidence is not a production deployment.

Proposed UI addition

Rust WASM owns intent, request state and rendering decisions. A native UI host keeps agent credentials off the browser and projects reports into a portable **ReportView**.

No browser UI or browser identity setup exists yet.

First experience to build

SELECT

Allowed node and project

INSPECT

One bounded request

EXPLORE

Tree and Rust declarations

UNDERSTAND

Completeness and failures

Use the references as a guide

Mission Control informs the prepare, worker fetch, response acceptance and render loop. Reuse the ownership pattern, not private source code or assumptions that its generator is drop-in.

Keep the first UI honest

Show a structural report, not a source editor, semantic compiler or live filesystem. No file-content endpoint, watcher, remote jobs or mutation API is part of this report.

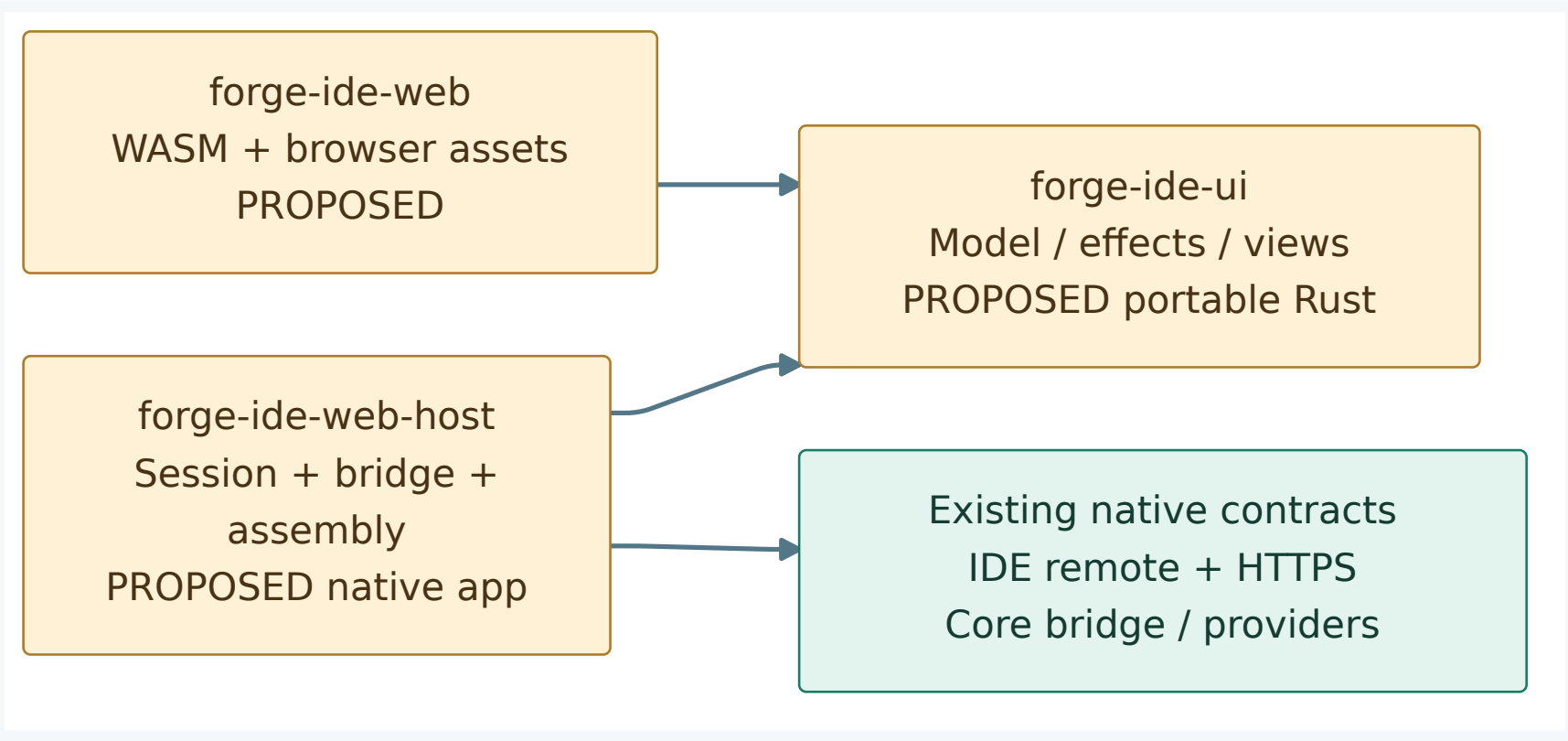
Decision to make before implementation Approve the Rust UI path, a scoped single-user host trust model, and the browser contract. This document does not authorize deployment or credential provisioning.

Implemented agent Current report model Full UI proposal

Add UI packages without reversing dependencies

Separate portable UI policy, the browser boundary and the native host. All new package names below are proposals. Existing agent, capability and core contracts stay independently usable.

Open full size SVG



Do not import the native capability into WASM

forge-ide inspects trusted projects with native filesystem and parser dependencies. The host validates **ProjectReport** and maps it to a bounded portable **ReportView**; a WASM build is a separate gate.

Preserve one command front door

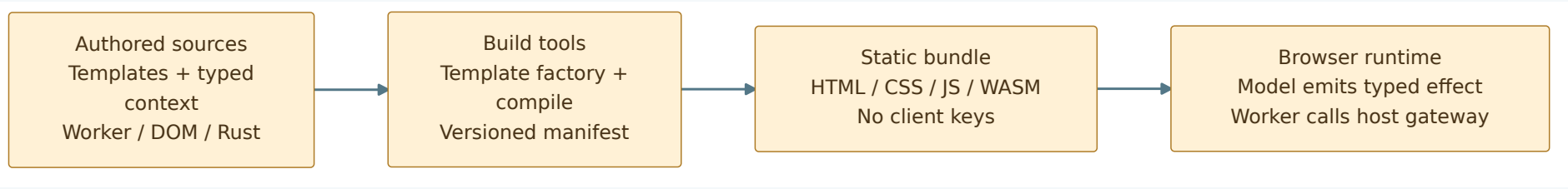
Static assets use GET. Commands share **POST /forge/v1/dispatch**. A bounded native owner runs the synchronous remote task and HTTPS objects; never call them on arbitrary async threads.

Existing composition Typed remote task Front door invariant

Keep build outputs separate from live state

Generate the stable HTML CSS and JavaScript shell once per build. Compile Rust UI logic to WASM. At runtime, load those assets and update selected DOM regions instead of regenerating the application.

Open full size SVG



Native build only

Publish asset hashes, protocol version and expected WASM exports in a manifest. Compile a real module and test the emitted bundle. Worker and DOM modules remain authored source.

Browser runtime only

Apply typed views to stable mount regions and preserve focus and expansion. Asset serving reads the bundle; it does not invoke the template factory for each response.

Reference tools are not interchangeable

Standalone UI references have separate static website and raw-ABI WASM paths. Mission Control uses an embedded legacy generator; it does not prove external codegen is already integrated.

Choose a small generation contract

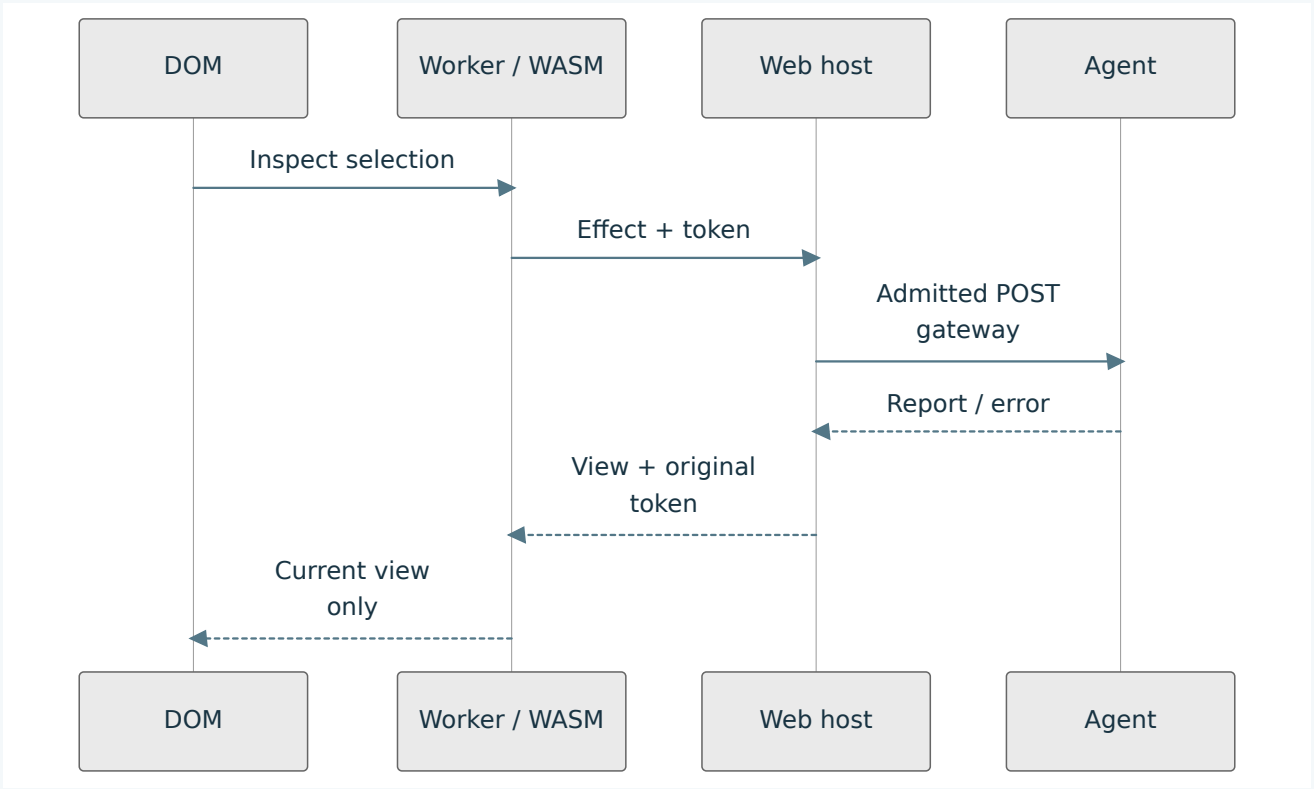
Whole-file code generation is not **forge-build** OUT_DIR resource embedding. Validate schema and template compatibility; keep generated files separate from handwritten state and effect logic.

Reference interpretation and build decision Current homelab template API

Route asynchronous effects through typed boundaries

One worker owns the Rust WASM model and fetch. It executes typed effects and lets Rust accept replies before publishing views. The native host separately admits browser sessions and invokes the agent.

Open full size SVG



Host admission is not agent admission

Host checks session, Host/Origin, CSRF and allowed selection. Its shared bridge invokes **RemoteInspectTask** with configured endpoint and budget; the browser supplies neither an arbitrary URL nor a path.

The agent checks authority again

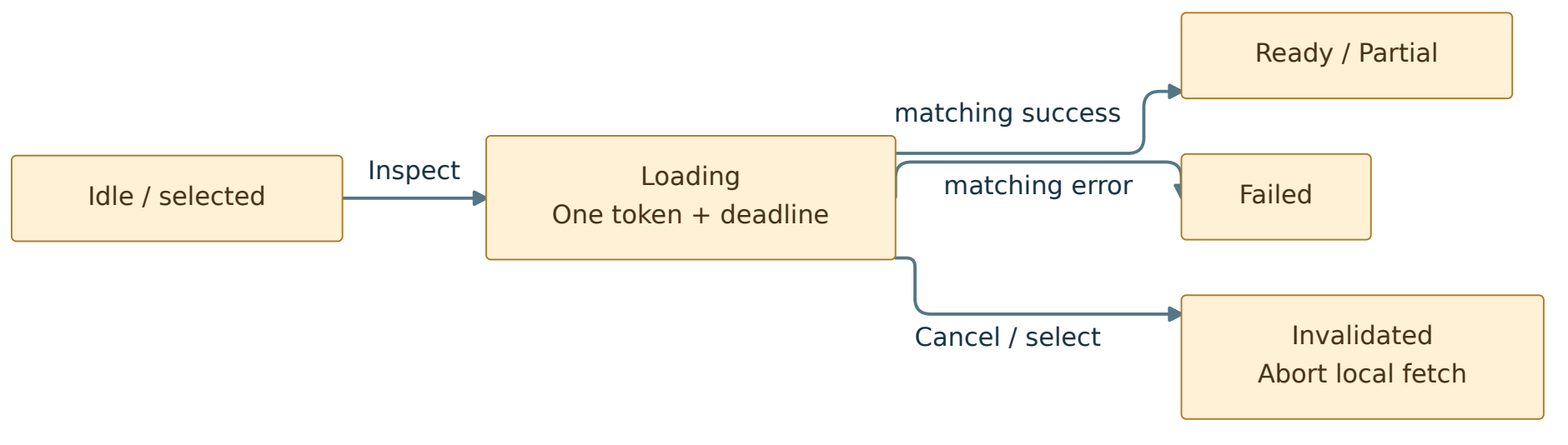
Agent checks mTLS subject, operation and project; its shared bridge guards inspection. Decode even HTTP 200 as report or error. Validate bounded **ReportView** and token before Rust publishes targeted updates.

Remote request and reply checks Agent dispatch policy Worker boundary

Make stale responses unable to replace current state

Treat selected node and project, the active request and accepted report as explicit Rust state. A response must match the session epoch, request ID and selection generation before changing the view.

Open full size SVG



A successful request can yield a partial report

Distinguish loading, ready, incomplete, denied, unavailable and invalid response. Preserve **complete=false**, excluded entries and diagnostics; do not display a green success badge for missing analysis.

Cancellation is a UI decision first

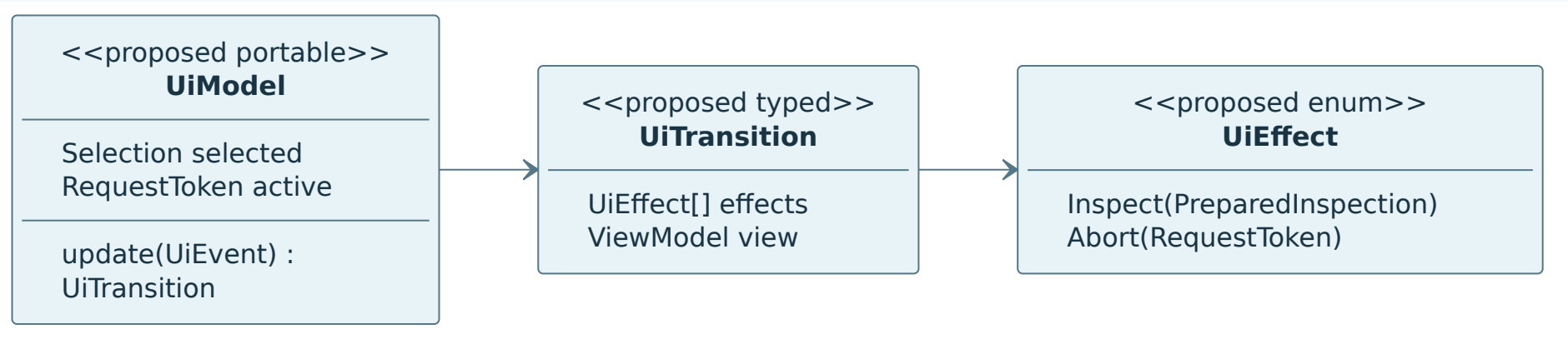
A new selection invalidates old work and its visible context. Abort fetch where possible, but ignore late replies regardless. Browser abort does not prove the agent stopped synchronous inspection.

Report completeness semantics Agent timeout and shutdown limits

Make state and resource ownership explicit

The worker owns the WASM instance, Rust view model and fetch. The small main-thread adapter applies DOM updates. Neither generated markup nor a second JavaScript store becomes the source of truth.

Open full size SVG



Mount and dispose deliberately

Mount once, attach delegated listeners and start bounded worker activity. On unmount, invalidate the generation, abort owned fetches, remove listeners and release handles. Workers never touch the DOM directly.

A raw ABI needs a tested ownership contract

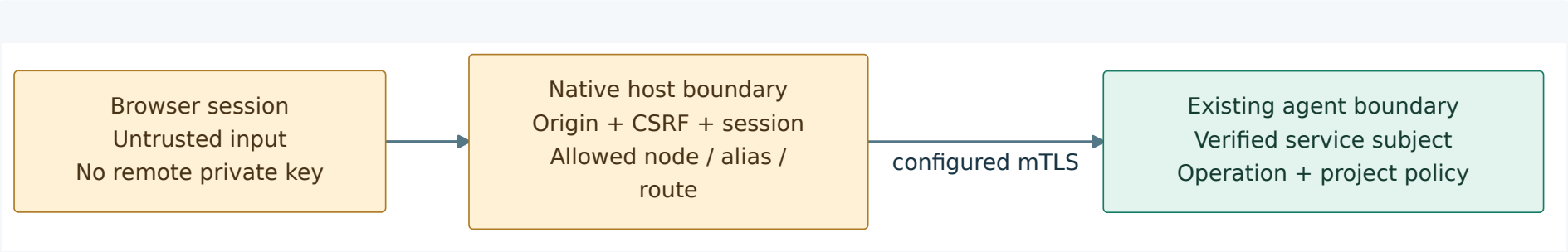
Version messages; define allocation, lengths and release. Copy bounded buffers before asynchronous use, refresh views after WASM memory growth, and never keep borrowed pointers across an await.

Web Worker ownership WASM memory growth Proposed interface contracts

Keep browser trust separate from agent trust

Recommend a same origin host bound to explicit loopback addresses for a trusted single-user first slice. Its protected session bootstrap is new work; mTLS keys never enter the browser.

Open full size SVG



Browser session

Protected bootstrap supplies only allowed node and project keys. A header or view label is not authority.

Native host

Effective grants intersect session, host configuration and agent service grants. No arbitrary relay or secret-bearing assets.

Agent service identity

The agent sees the host TLS subject, not the browser user. Its operation and project policy still applies.

The host must not be an unrestricted proxy

Allowlist session, operation, node and project before forwarding. Use a narrowly scoped service identity and preserve agent checks. This is not end-user delegation; multi-user isolation needs a separate design.

Browser protections remain mandatory

Require session authentication, expected Host/Origin and CSRF checks, secure cookie policy and no permissive credentialed CORS. Render text safely, test CSP, and keep reports and secrets out of logs and persistent browser storage.

Current agent authority boundary CSRF and Origin guidance Trust model and open decisions

Overview Layout Build Request State Ownership **Security** Plan

7 / 8

GitHub Proposal Diagram gallery Front door skill

Ship a narrow browser inspection slice

Build the smallest useful path from an allowed project selection to a validated structural report. Each stage needs tests for rejection, stale data and cleanup before expanding the surface area.

01

Pure state and view

Portable model and fixture tests

02

Generated shell

Stable mounts and ABI smoke test

03

Scoped native host

Session policy and agent adapter

04

Browser integration

Real interaction and failure tests

Acceptance evidence required

Portable boundary	WASM builds without native filesystem TLS or parser crates; default core profiles stay unchanged.
Generator and ABI	Deterministic outputs, version mismatch refusal, balanced buffer release and mount cleanup.
State and rendering	Out-of-order and duplicate replies, selection changes, partial reports, escaping and bounded DOM work.
Authority and transport	Unauthenticated or cross-origin request, denied node/project, forged identity, wrong peer and oversized reply.
User experience	Keyboard tree navigation, visible focus, accessible status, empty/error states and slow network behavior.
End to end	Real localhost browser / host / agent flow with ephemeral identities; late replies never repaint.

Deliberately later Source text editing, writes, language-server features, remote jobs, live subscriptions, multi-user delegation and alternate carriers. No runtime code, deployment or credential provisioning is delivered by this report.

Full proposal and decision gates Rebuild this report

Overview Layout Build Request State Ownership Security **Plan** 8 / 8

GitHub Proposal Diagram gallery Front door skill